КОМБИНАТОРНЫЕ МЕТОДЫ РЕШЕНИЯ ОПТИМИЗАЦИОННЫХ ЗАДАЧ

Цель: освоение навыков решения оптимизационных задач комбинаторными методами.

Задачи:

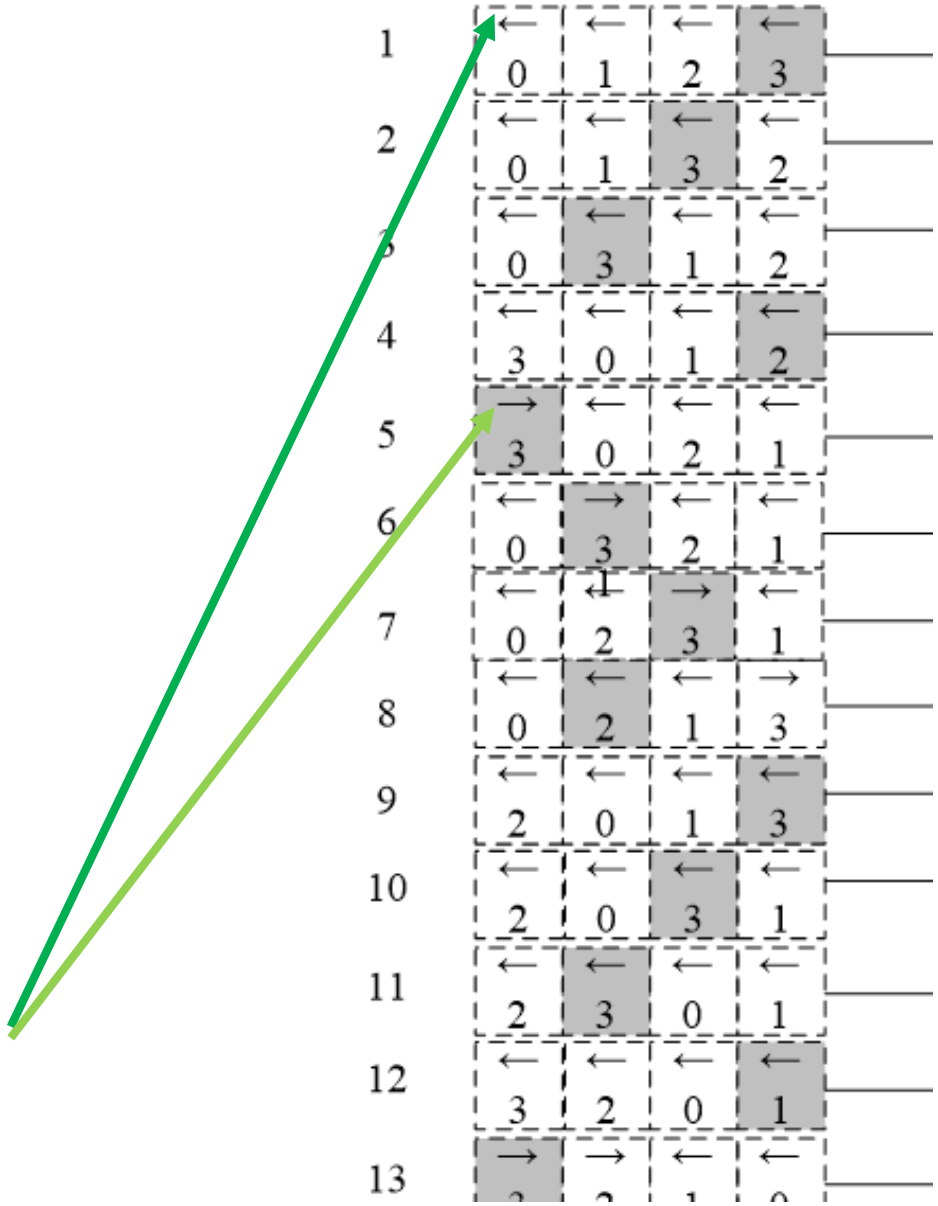
- изучение особенностей применения комбинаторных алгоритмов решения оптимизационных задач;
- изучение теоретических основ комбинаторных алгоритмов;
- практическое применение алгоритмов для решения известных оптимизационных задач на языке программирования C++.

Генерация перестановок

Алгоритм Джонсона – Троттера

Этот алгоритм является наиболее известным методом построения множества P_X всех перестановок конечного множества X.

Алгоритм подразумевает, что все элементы множества X можно единственным способом перечислить в порядке возрастания. Отметим, что для конечного множества такой порядок всегда можно установить. Каждый элемент исходного множества X помечается специальным символом – стрелкой, которая может быть направлена влево или вправо.



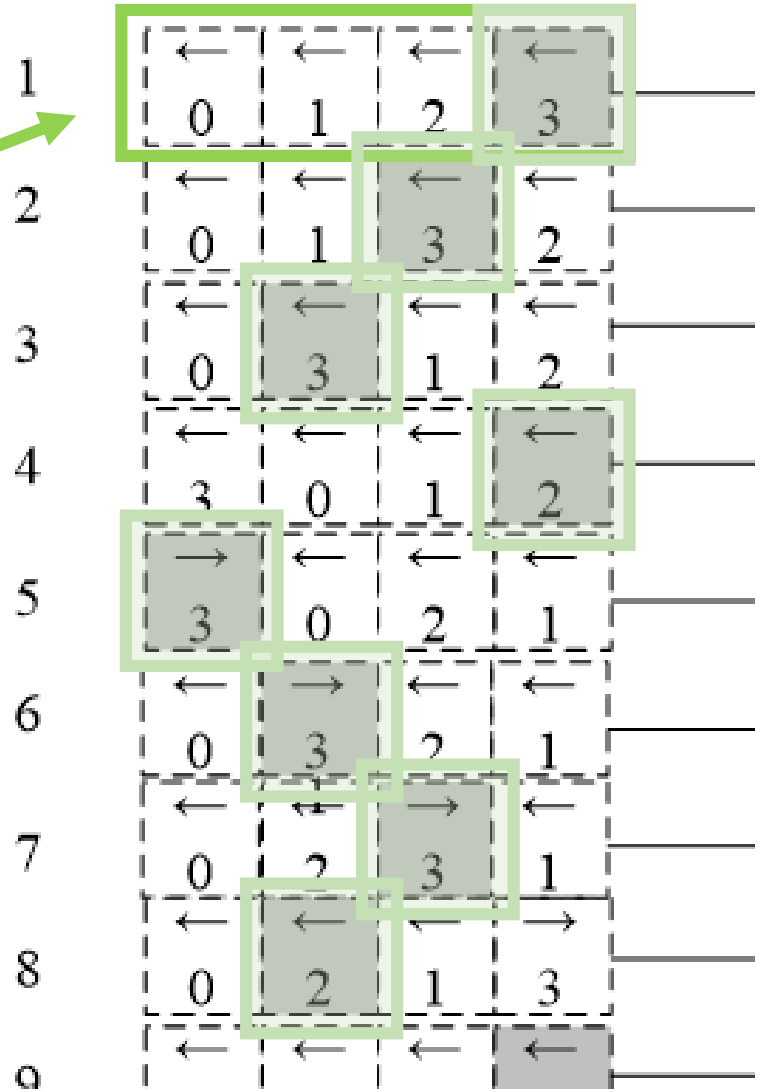
Генерация перестановок

Алгоритм Джонсона – Троттера

Первая перестановка в алгоритме Джонсона – Троттера выглядит следующим образом:

$$\overleftarrow{1}, \overleftarrow{2}, \overleftarrow{3}, \ldots, \overleftarrow{n}.$$

В алгоритме используется понятие мобильного элемента. Элемент последовательности элементов множества ***X*** называется мобильным, если соответствующая ему стрелка указывает на меньший соседний элемент.



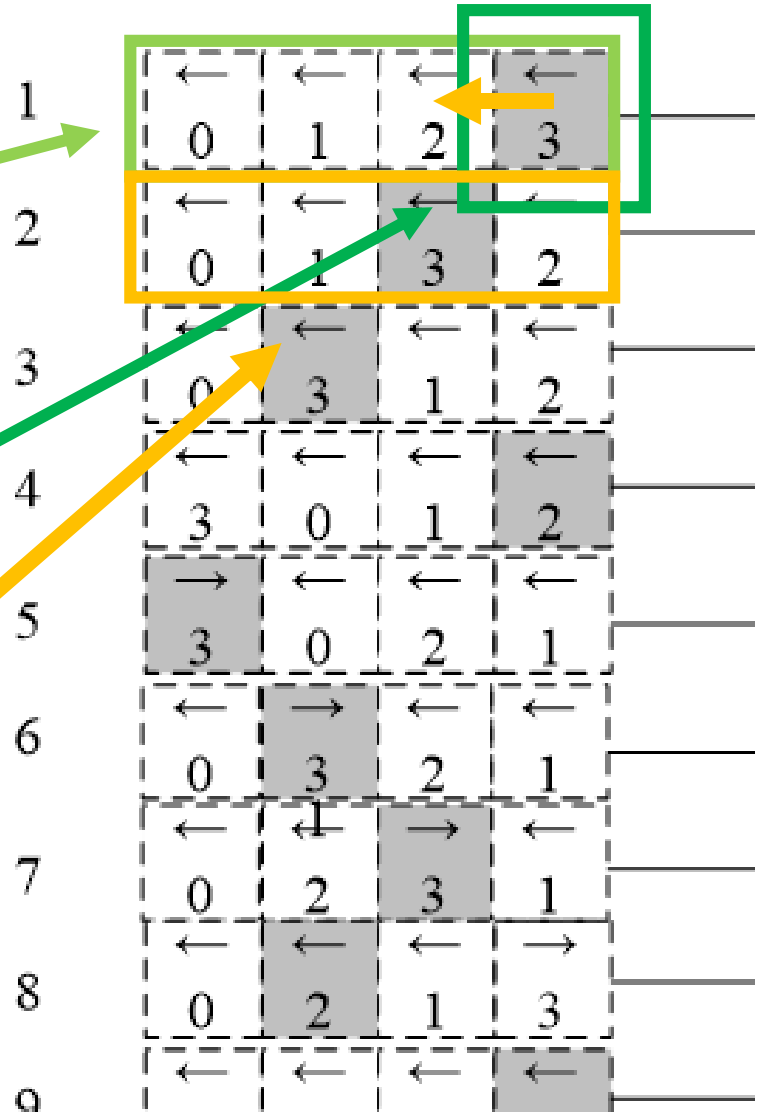
В первой перестановке все элементы кроме самого левого являются мобильными.

Генерация перестановок

Алгоритм Джонсона – Троттера

Построение множества всех перестановок с помощью алгоритма Джонсона – Троттера сводится к следующей процедуре:

1. Построить первую перестановку. Первая перестановка – это последовательность всех элементов множества ***X*** перечисленных в порядке возрастания. Стрелки всех элементов последовательности направлены влево.
2. Найти наибольший мобильный элемент в текущей перестановке. Если в последовательности нет мобильного элемента, то построены все перестановки элементов множества ***X*** – алгоритм закончил свою работу.
3. Поменять местами наибольший мобильный элемент и элемент, на который указывает стрелка наибольшего мобильного элемента.
4. Найти все элементы, большие, чем наибольший мобильный элемент (если они есть) и изменить их стрелки на противоположное направление.
5. Перейти к пункту 2.



Генерация перестановок

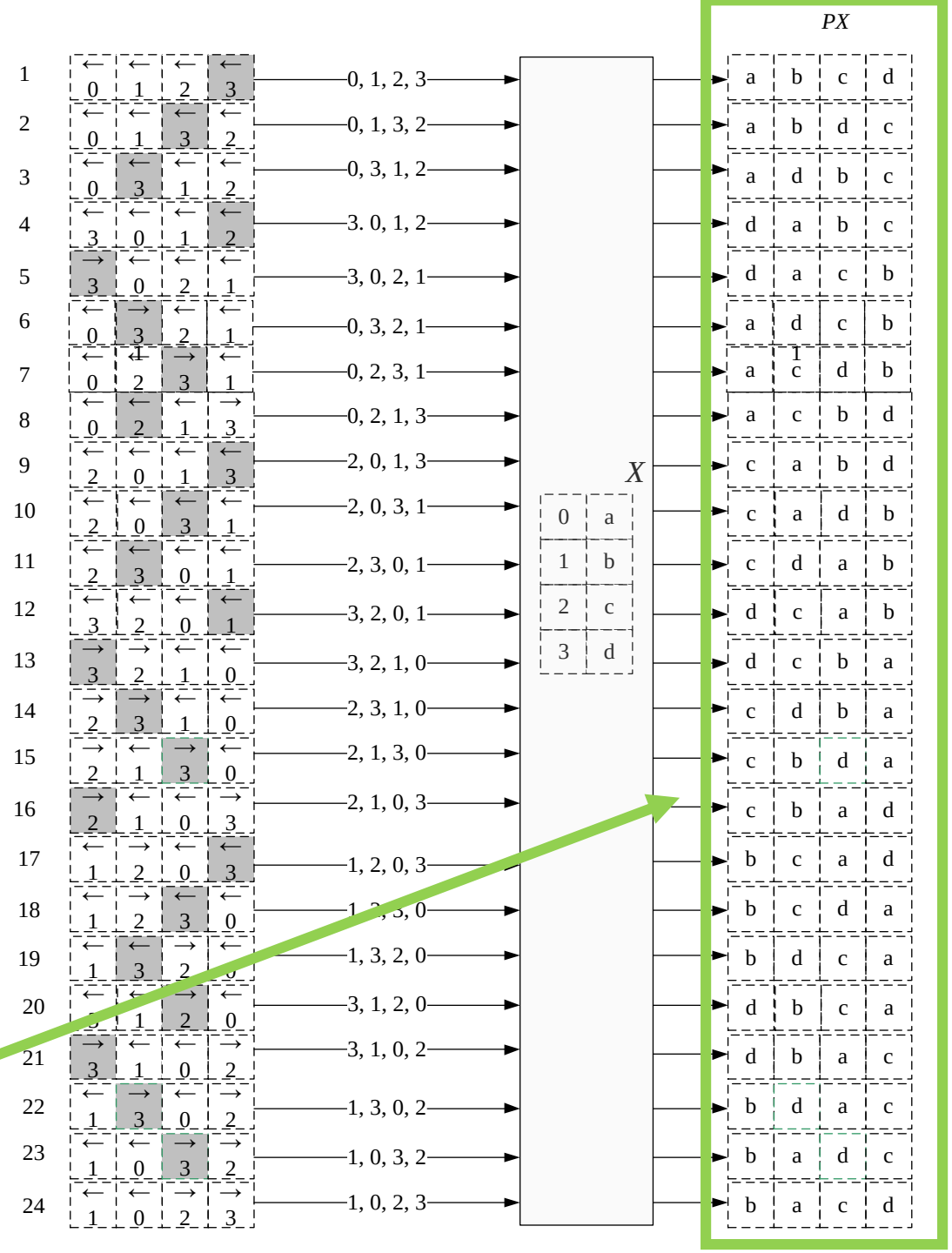
Алгоритм Джонсона – Троттера

Количество массивов составляет:

$$|P_A| = P_n = n!$$

$$P_4 = 4! = 24.$$

Самый правый столбец (обозначен ***P**_X*) – множество всех перестановок элементов ***X***={**a, b, c, d**}. Перестановки формируются как результат индексации массива ***X.***



Генерация перестановок
Алгоритм Джонсона – Троттера

Генератор реализован в виде структуры **permutation**.

Структура имеет один конструктор. С помощью параметра конструктору передается размерность исходного множества.

Состояние генератора определяется значениями четырех переменных:

n (количество элементов в исходном множестве),
sset (указатель на массив индексов),
dart (указатель на массив стрелок),
np (номер текущей перестановки).

Генерация перестановок

Алгоритм Джонсона – Троттера

```
struct  permutation
 {
    const static bool L = true;  // левая стрелка
    const static bool R = false; // правая стрелка
    short  n,            // количество элементов исходного множества
        *sset;          // массив индексов текущей перестановки
    bool  *dart;          // массив  стрелок (левых-L и правых-R)
    permutation (short n = 1);
void reset();             // сбросить генератор, начать сначала
__int64 getfirst();         // сформировать первый массив индексов
__int64 getnext();          // сформировать случайный массив индексов
short ntx(short i);         // получить i-й элемент масива индексов
unsigned __int64 np;        // номер перествновки 0,... count()-1
unsigned __int64 count() const;  // вычислить общее кол. перестановок
 };
```

Генерация перестановок
Алгоритм Джонсона – Троттера

Все переменные инициализируются в конструкторе. Значение **np** увеличивается на единицу после генерации очередной перестановки, значение остальных переменных остается неизменным.

```
permutation::permutation(short n)
{
   this->n = n;
   this->sset = new short[n];
   this->dart = new bool[n];
   this->reset();
};
```

Генерация перестановок
Алгоритм Джонсона – Троттера

Элементы массивов **sset** и **dart** меняются при каждом цикле работы генератора в соответствии с алгоритмом Джонсона –Троттера. Функция **getfirst** не имеет параметров и предназначена для формирования первой перестановки, которая представляет собой упорядоченную последовательность **n** неотрицательных целых чисел.

```
__int64  permutation::getfirst()
 {
   this->np = 0; //#p
   for (int i = 0; i < this->n; i++)
        {this->sset[i] = i; //индексы
       this->dart[i] = L;}; //стрелки
   return  (this->n > 0)?this->np:-1;
};
```

Алгоритм Джонсона – Троттера

Функция **reset** позволяет сбросить текущее состояние генератора для того, чтобы начать его работу сначала. Работа функции сводится к вызову функции **getfirst**. Функция **reset** реализована главным образом для унификации интерфейсов всех генераторов.

```
void  permutation::reset()
 {  this->getfirst(); };
```

Генерация перестановок

Алгоритм Джонсона – Троттера

Функция **getnext** формирует массив индексов следующей перестановки и увеличивает значение переменной **np** на единицу. При каждом вызове функции в массиве **sset** отыскивается максимальный мобильный элемент и элемент, на который указывает его стрелка, эти элементы меняются местами.

```
__int64  permutation::getnext()
 {
   __int64 rc = - 1;
   short maxm = NINF, idx = -1;
   for(int i = 0; i < this->n; i++){
   if ( i > 0 && this->dart[i] == L &&
  this->sset[i] > this->sset[i-1] &&
  maxm < this->sset[i])
maxm = this->sset[idx = i];

   if ( i < (this->n-1)&&
  this->dart[i] == R &&
  this->sset[i] > this->sset[i+1]&&
          maxm < this->sset[i])  maxm = this-
>sset[idx = i]; };
```

Генерация перестановок
Алгоритм Джонсона – Троттера

Если существуют элементы **sset**, большие, чем найденный мобильный, то соответствующие им стрелки инвертируются.

```
 if (idx  >= 0)
{  std::swap(this->sset[idx],
   this->sset[idx+(this->dart[idx]== L?-1:1)]);
        std::swap(this->dart[idx],
        this->dart[idx+(this->dart[idx]== L?-1:1)]);

for (int i = 0; i < this->n; i++)
     if (this->sset[i] > maxm) this->dart[i] = !this->dart[i];
  rc = ++this->np;}
     return rc;};
```

Генерация перестановок
Алгоритм Джонсона – Троттера

Функция **ntx** возвращает значение элемента массива индексов по индексу этого элемента и служит для сокращения записи при переборе элементов массива.

```
short permutation::ntx(short i){return  this->sset[i];};
```

Функция **count** вычисляет и возвращает общее количество перестановок **n** элементов множества.

```
unsigned __int64 permutation::count() const  {return
fact(this->n); };
```

```
// --- Main
#include "stdafx.h"
#include <iostream>
#include "Combi.h"
#include <iomanip>
int _tmain(int argc, _TCHAR* argv[])
{
 setlocale(LC_ALL, "rus");
 char  AA[][2]= {"A", "B", "C", "D"};
 std::cout<<std::endl<<" --- Генератор перестановок ---";
 std::cout<<std::endl<<"Исходное множество: ";
 std::cout<<"{ ";
 for (int i = 0; i < sizeof(AA)/2; i++)

      std::cout<<AA[i]<<((i< sizeof(AA)/2-1)?", ":" ");
 std::cout<<"}";
```

```
std::cout<<std::endl<<"Генерация перестановок ";
 combi::permutation p(sizeof(AA)/2);
 __int64  n  = p.getfirst();
 while (n >= 0)
 {
 std::cout<<std::endl<<std::setw(4)<< p.np <<": { ";
 for (int i = 0; i < p.n; i++)
 std::cout<<AA[p.ntx(i)]<<((i< p.n-1)?", ":" ");
 std::cout<<"}";
 n = p.getnext();
 };
 std::cout<<std::endl<<"всего: " << p.count()<<std::endl;
 system("pause");
 return 0;
}
```

g:\Documents and Settings\smw\Мои докуме...

```
 --- Генератор перестановок ---
Исходное множество: < A, B, C, D >
Генерация перестановок
    0: < A, B, C, D >
    1: < A, B, D, C >
    2: < A, D, B, C >
    3: < D, A, B, C >
    4: < D, A, C, B >
    5: < A, D, C, B >
    6: < A, C, D, B >
    7: < A, C, B, D >
    8: < C, A, B, D >
    9: < C, A, D, B >
   10: < C, D, A, B >
   11: < D, C, A, B >
   12: < D, C, B, A >
   13: < C, D, B, A >
   14: < C, B, D, A >
   15: < C, B, A, D >
   16: < B, C, A, D >
   17: < B, C, D, A >
   18: < B, D, C, A >
   19: < D, B, C, A >
   20: < D, B, A, C >
   21: < B, D, A, C >
   22: < B, A, D, C >
   23: < B, A, C, D >
всего: 24
Для продолжения нажмите любую клавишу . . .
```

Решение задачи коммивояжера с использованием генератора перестановок

Коммивояжер (бродячий торговец) должен найти минимальный кольцевой маршрут обхода n городов. Расстояние d_{ij} между каждой парой $\langle i,\ j\rangle,\ i,\ j=\overline{0, n-1}$ городов считается известным.

Математическая модель задачи

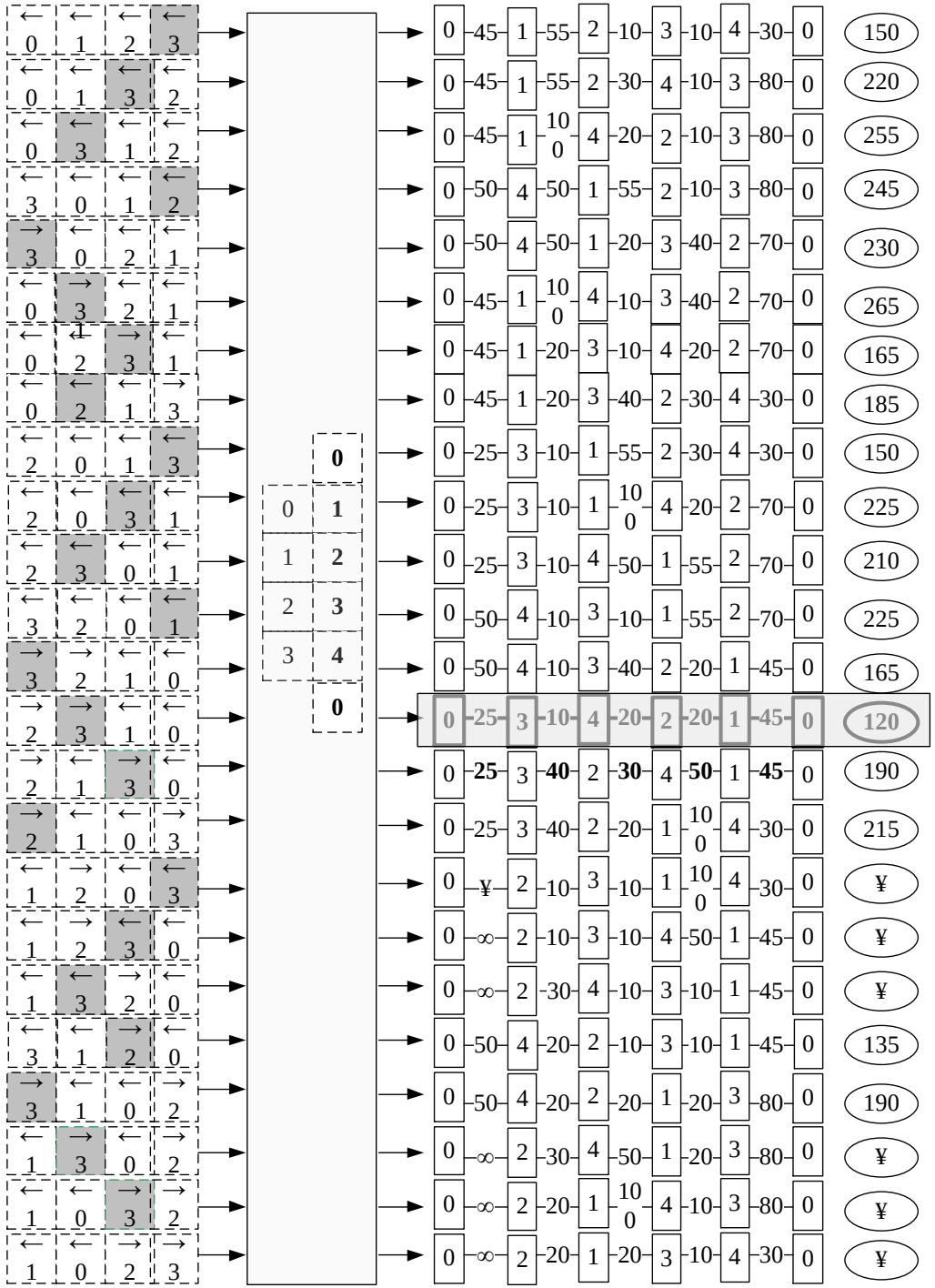
$$d_{0,k_1}+\sum_{i=1}^{n-1} d_{k_i,k_{i+1}}+d_{k_{n-1},0}\to \min,$$

$$k_i\in\{1,2,\ldots,n-1\},\ \forall(i\neq j)\quad k_i\neq k_j,\ \ i=\overline{0, n-1},$$

Решение задачи коммивояжера с использованием генератора перестановок

Решением задачи будет вектор $(k_1, k_2, ..., k_m)$. Каждый элемент этого вектора может принимать целое значение из отрезка $[1, n-1]$. При этом все значения k_i должны быть разными.

Решение задачи сводится к генерации всех допустимых векторов $(k_1, k_2, ..., k_m)$, вычислению функции $d_{0,k_1} + \sum_{i=1}^{n-1} d_{k_i,k_{i+1}} + d_{k_{n-1},0}$ и выбору вектора, соответствующего минимальному значению функции.



Расстояние между городами задается матрицей $\boldsymbol{A}$. Элемент $\boldsymbol{a_{ij}}$ матрицы $\boldsymbol{A}$ определяет расстояние между городами i и j, где

$$i, j = \overline{0, 4}.$$

$$A = (a_{ij})_{5\times5} = \begin{pmatrix} 0 & 45 & \infty & 25 & 50 \\ 45 & 0 & 55 & 20 & 100 \\ 70 & 20 & 0 & 10 & 30 \\ 80 & 10 & 40 & 0 & 10 \\ 30 & 50 & 20 & 10 & 0 \end{pmatrix}.$$

Факт отсутствия пути из города i в город j обозначается значением ∞ (бесконечность) соответствующего элемента.

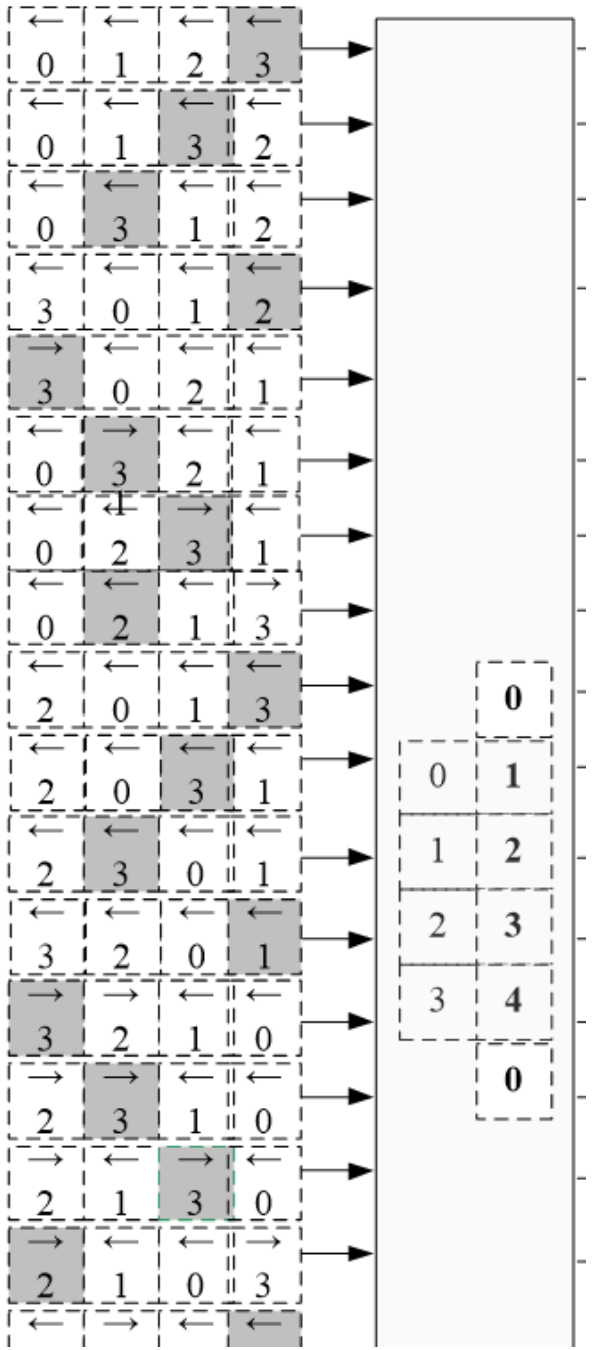
Решение задачи коммивояжера с использованием генератора перестановок

Замечания:

- При построении оптимального маршрута коммивояжера выбор стартового (он же является и конечным, так как маршрут кольцевой) города никак не влияет на конечный результат.
- Если задано ***n*** городов, то перебор следует осуществлять только для ***n***-1 городов, поскольку стартовый город можно зафиксировать.

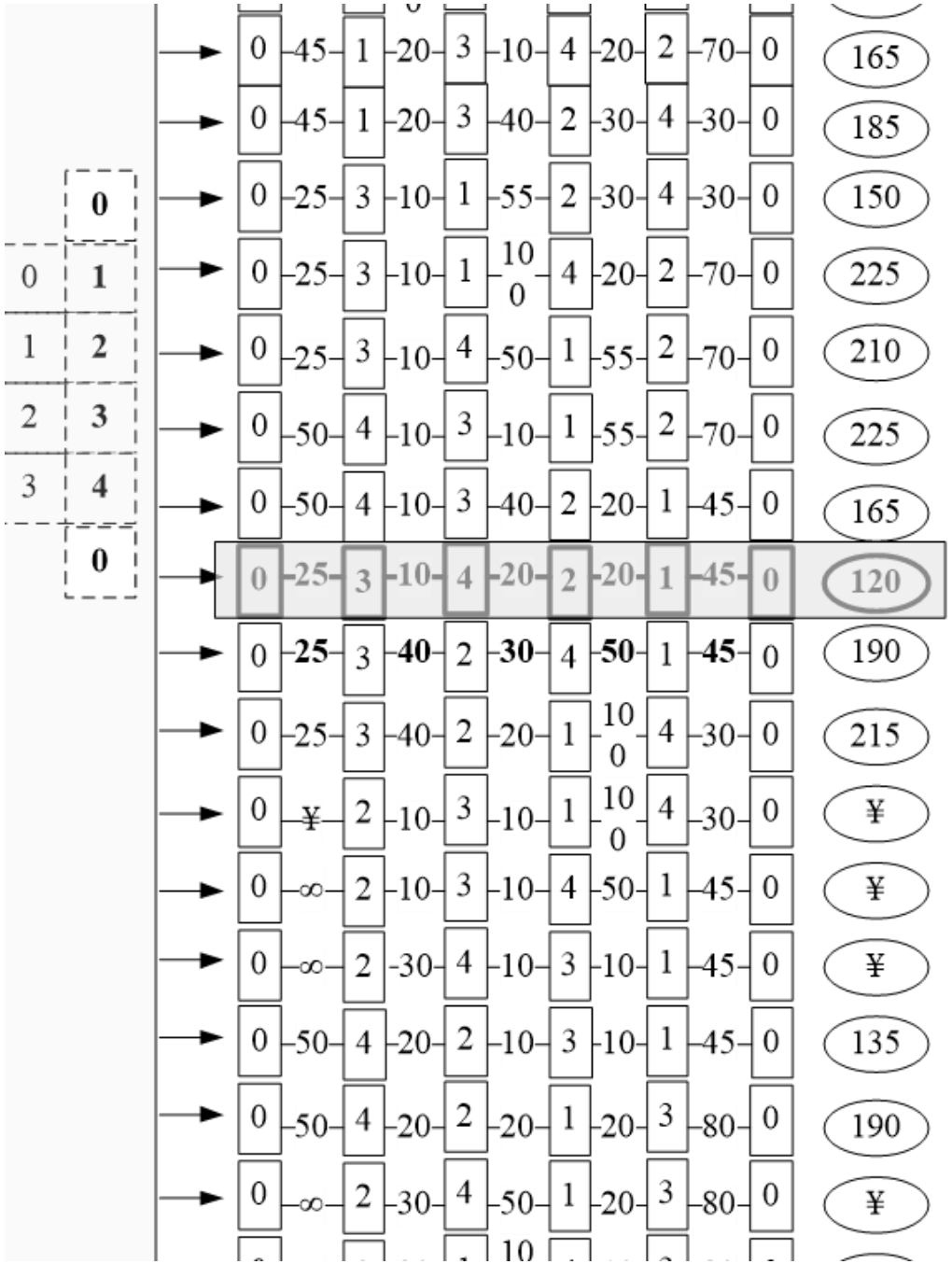
Решение задачи коммивояжера с использованием генератора перестановок

Левая часть схемы практически идентична левой части схемы на генератора перестановок. Главное отличие заключается в том, что в качестве исходного массива выбран {0, 1, 2, 3, 4, 0} Причем перестановкам подлежит только внутренняя, заключенная между нулями, часть исходного массива.



Решение задачи коммивояжера с использованием генератора перестановок

В правой части схемы изображены все возможные кольцевые маршруты, которые образованы из исходного массива путем перестановок всех, кроме обрамляющих нулей, элементов. Количество маршрутов 4!=24 равно количеству перестановок из четырех городов. Для каждого маршрута в округлой рамке указана длина. Длина кольцевых маршрутов, для которых не могут быть построены в силу отсутствия пути хотя бы между одной парой городов, на схеме обозначена символом ∞.



Решение задачи коммивояжера с использованием генератора перестановок

Пример реализации на C++ функции **salesman**, вычисляющей оптимальный кольцевой маршрут коммивояжера.

Функция имеет два входных параметра: **n** (количество городов) и **d** (двумерный массив размерностью **n** на **n**, содержащий элементы матрицы расстояний), а также один возвращаемый параметр **r** (массив размерностью **n**, содержащий оптимальный маршрут).

В функции **salesman** применяется генератор перестановок (**combi::permutation**).

Решение задачи коммивояжера с использованием генератора перестановок

```
int salesman (
 int n,        // [in]  количество городов
 const int *d,  // [in]  массив [n*n] расстояний
 int *r        // [out] массив [n] маршрут 0 x x x x
  );
```

Решение задачи коммивояжера с использованием генератора перестановок

Кроме того функция **salesman** вызывает шесть вспомогательных функций: **indx** (формирование перестановки городов на основе массива идексов), **distance** (вычисление длины кольцевого маршрута), **copypath** (копирование маршрута), **source** (формирование исходного массива), **firstpath** (формирование первого маршрута) и **sum** (суммирование двух чисел с учетом того, что одно из них может быть равно бесконечности).

```
void indx(int n, int *r, const int *s, const short *ntx)
 { for (int i = 1; i < n; i++)  r[i] = s[ntx[i-1]];}
```

Решение задачи коммивояжера с использованием генератора перестановок

Кроме того функция **salesman** вызывает шесть вспомогательных функций: **indx** (формирование перестановки городов на основе массива идексов), **distance** (вычисление длины кольцевого маршрута), **copypath** (копирование маршрута), **source** (формирование исходного массива), **firstpath** (формирование первого маршрута) и **sum** (суммирование двух чисел с учетом того, что одно из них может быть равно бесконечности).

```
int distance(int n, int *r, const int *d)      // длина маршрута
 {
   int rc = 0;
   for (int i = 0; i < n-1; i++) rc = sum(rc, d[r[i]*n+r[i+1]]);
   return  sum (rc, d[r[n-1]*n + 0]);    //+ последняя дуга (n-1,0) };
```

Решение задачи коммивояжера с использованием генератора перестановок

Кроме того функция **salesman** вызывает шесть вспомогательных функций: **indx** (формирование перестановки городов на основе массива идексов), **distance** (вычисление длины кольцевого маршрута), **copypath** (копирование маршрута), **source** (формирование исходного массива), **firstpath** (формирование первого маршрута) и **sum** (суммирование двух чисел с учетом того, что одно из них может быть равно бесконечности).

```
void  copypath(int n, int *r1, const int *r2)  //
копировать маршрут
 {  for (int i = 0; i <  n; i++)  r1[i] = r2[i]; };
```

Решение задачи коммивояжера с использованием генератора перестановок

Кроме того функция **salesman** вызывает шесть вспомогательных функций: **indx** (формирование перестановки городов на основе массива идексов), **distance** (вычисление длины кольцевого маршрута), **copypath** (копирование маршрута), **source** (формирование исходного массива), **firstpath** (формирование первого маршрута) и **sum** (суммирование двух чисел с учетом того, что одно из них может быть равно бесконечности).

```
int*  source(int n)   // формирование исходного
массива 1,2,..., n-1
 {
   int *rc = new int[n-1];
   for (int i = 1; i < n; i++) rc[i-1] = i;
   return rc;
 };
```

Решение задачи коммивояжера с использованием генератора перестановок

Кроме того функция **salesman** вызывает шесть вспомогательных функций: **indx** (формирование перестановки городов на основе массива идексов), **distance** (вычисление длины кольцевого маршрута), **copypath** (копирование маршрута), **source** (формирование исходного массива), **firstpath** (формирование первого маршрута) и **sum** (суммирование двух чисел с учетом того, что одно из них может быть равно бесконечности).

```
int* firstpath(int n) // формирование 1го маршрута 0,1,2,..., n-1, 0
 {
   int *rc = new int[n+1]; rc[n] = 0;
   for (int i = 0; i < n; i++) rc[i] = i;
   return rc;
 };
```

Решение задачи коммивояжера с использованием генератора перестановок

Кроме того функция **salesman** вызывает шесть вспомогательных функций: **indx** (формирование перестановки городов на основе массива идексов), **distance** (вычисление длины кольцевого маршрута), **copypath** (копирование маршрута), **source** (формирование исходного массива), **firstpath** (формирование первого маршрута) и **sum** (суммирование двух чисел с учетом того, что одно из них может быть равно бесконечности).

```
int sum (int x1, int x2) // суммирование с учетом бесконечности
 { return (x1 == INF || x2 == INF)? INF: (x1 + x2); };
```

Решение задачи коммивояжера с использованием генератора перестановок

Функция **salesman** в цикле генерирует все возможные кольцевые маршруты, вычисляет для каждого маршрута длину (функция **distance**), фиксирует оптимальный маршрут (функция **copypath**) и возвращает длину оптимального пути или значение **INF**, что обозначает отсутствие кольцевых маршрутов.

```
int salesman (
 int n,        // [in]  количество городов
 const int *d,  // [in]  массив [n*n] расстояний
 int *r        // [out] массив [n] маршрут 0 x x x x
 )
{int *s = source(n),  *b = firstpath(n), rc = INF, dist = 0;
 combi::permutation p(n-1);
 int k = p.getfirst();
 while (k >= 0)  // цикл генерации перестановок
  {
    indx(n, b, s, p.sset);       // новый маршрут
    if ((dist = distance(n,b,d)) < rc)
 { rc = dist; copypath(n,r,b); }
    k = p.getnext();  };
 return rc;   }
```

```
/// --- main
#include "stdafx.h"
#include <iostream>
#include <iomanip>
#include "Salesman.h"
#define N 5
int _tmain(int argc, _TCHAR* argv[])
{
 setlocale(LC_ALL, "rus");
 int d[N][N] = { //0   1    2    3     4
            {  0,  45, INF,  25,   50},    //  0
            { 45,   0,  55,  20,  100},    //  1
            { 70,  20,   0,  10,   30},    //  2
            { 80,  10,  40,   0,   10},    //  3
            { 30,  50,  20,  10,    0}};   //  4
int r[N];                    // результат
int s = salesman (
          N,         // [in]  количество городов
     (int*)d,         // [in]  массив [n*n] расстояний
          r          // [out] массив [n] маршрут 0 x x x x
                          );
```

```
std::cout<<std::endl<<"-- Задача коммивояжера -- ";
 std::cout<<std::endl<<"-- количество  городов: "<<N;
 std::cout<<std::endl<<"-- матрица расстояний : ";
 for(int i = 0; i < N; i++)
  {
    std::cout<<std::endl;
    for (int j = 0; j < N; j++)

if (d[i][j]!= INF) std::cout<<std::setw(3)<<d[i][j]<< " ";

else std::cout<<std::setw(3)<<"INF"<<" ";
  }
 std::cout<<std::endl<<"-- оптимальный маршрут: ";
 for(int i = 0; i < N; i++) std::cout<<r[i]<<"-->"; std::cout<<0;
 std::cout<<std::endl<<"-- длина маршрута     : "<<s;
 std::cout<<std::endl;
 system("pause");
return 0;
}
```

g:\Documents and Settings\smw\Мои документы\...

```
-- Задача коммивояжера --
-- количество  городов: 5
-- матрица расстояний :
  0   45 INF   25   50
 45    0   55   20  100
 70   20    0   10   30
 80   10   40    0   10
 30   50   20   10    0
-- оптимальный маршрут: 0-->3-->4-->2-->1-->0
-- длина маршрута     : 120
Для продолжения нажмите любую клавишу . . .
```

Решение задачи коммивояжера с использованием генератора перестановок

Также можно оценить продолжительность выполнения расчетов алгоритмом.

В программе применяются функции **auxil::start** и **auxil::iget**, позволяющие сгенерировать расстояния между городами случайным образом, которые берутся из первой лабораторной работы.

```
#define SPACE(n) std::setw(n)<<" "
#define N 12
int _tmain(int argc, _TCHAR* argv[])
{
int d[N*N+1], r[N];
 auxil::start();
 for(int i = 0; i <= N*N; i++) d [i] =
auxil::iget(10,100);
clock_t t1, t2;
 for (int i = 7; i <= N; i++)
 { t1 = clock();
 salesman (i, (int*)d, r);
 t2 = clock();

std::cout<<std::endl<<SPACE(7)<<std::setw(2)<<i<
<SPACE(15)<<std::setw(5)<<(t2-t1);}
 std::cout<<std::endl;
 system("pause"); return 0;}
```

g:\Documents and Settings\smw\Мои докуме...

```
-- Задача коммивояжера --
-- количество ------ продолжительность --
     городов              вычисления
        7                      0
        8                      0
        9                     31
       10                    313
       11                   3375
       12                  39234
Для продолжения нажмите любую клавишу . . .
```

Сложность $O((n-1)!)$

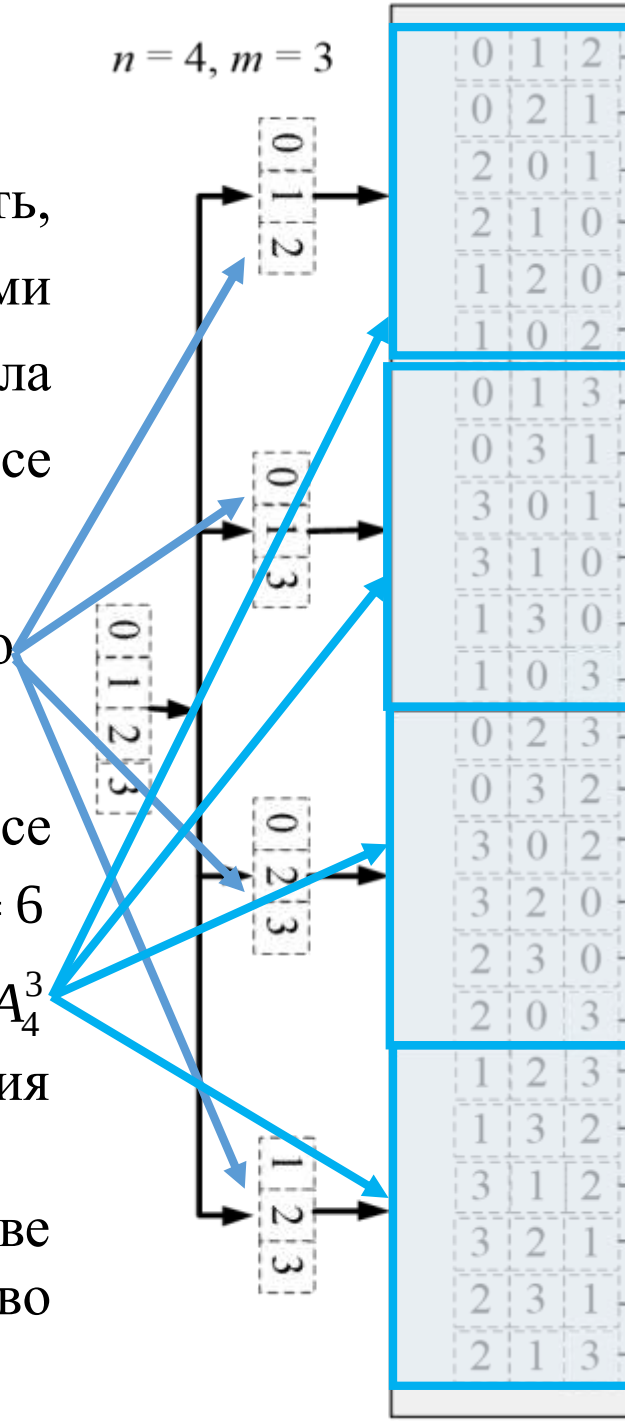
Генерация размещений

Множество размещений $A_{X,m}$ из $|X| = n$ по m можно получить, перестановкой элементов всех сочетаний множества $C_{X,m}$. Другими словами, для получения множества размещений $A_{X,m}$, требуется сначала сгенерировать все сочетания по m элементов из множества X, а затем все перестановки элементов для каждого сочетания.

1. Для множества $\{0, 1, 2\ 3\}$ формируется множество всех сочетаний по три элемента. Таких сочетаний будет $C_4^3 = 4$.

2. Для элементов каждого сочетания c_i, $i = \overline{1, 4}$ генерируется все перестановки P_{c_i}. Каждому сочетанию будет соответствовать $P_3 = 6$ перестановок. Таким образом, всего будет построено $C_4^3 \cdot P_3 = A_4^3$ перестановок, которые в совокупности представляют собой все размещения множества $\{0, 1, 2, 3\}$.

3. Применяя полученные на предыдущем шаге размещения в качестве индексов для элементов множества $X = \{a, b, c, d\}$, формируется множество $A_{X,3}$ всех размещений по 3.



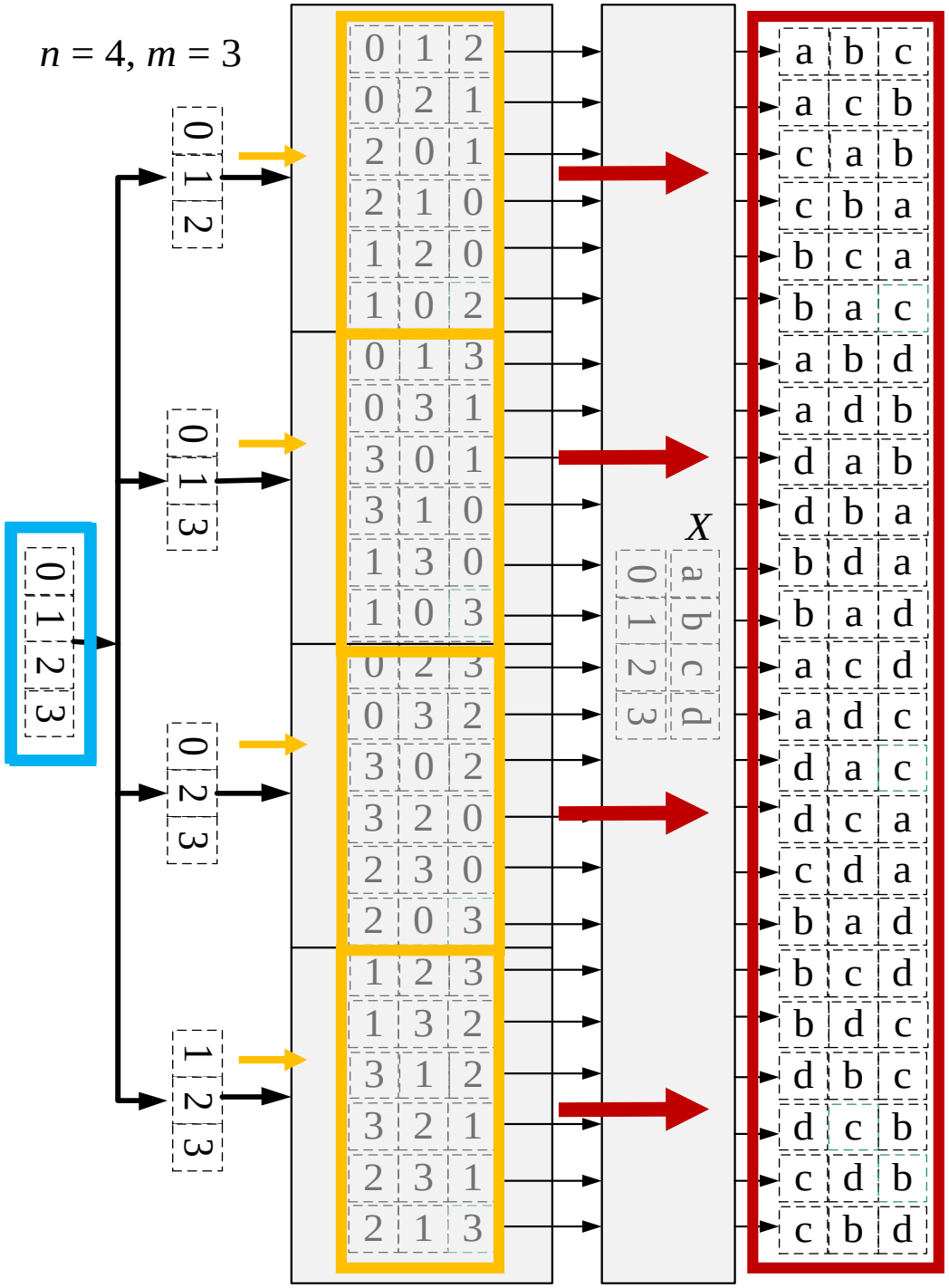
Генерация размещений

Схема построения множества размещений $A_{X,3}$ из элементов множества $X=\{a, b, c, d\}$.

Для того чтобы проследить этапы генерации множества $A_{X,3}$ на схеме, ее следует рассматривать слева направо. В крайней левой позиции изображено множество {0, 1, 2, 3}, на основе которого формируются четыре сочетания.

Затем каждое сочетание рассматривается как отдельное множество, состоящее из трех элементов. Для каждого множества формируется по 3!=6 перестановок. В итоге в третьем слева столбце на схеме отображены 4·6=24 перестановки множества {0, 1, 2, 3}.

На последнем этапе формируется множество $A_{X,3}$ всех перестановок элементов множества $X=\{a, b, c, d\}$. Элементы множества $A_{X,3}$ отображены в крайнем справа столбце.



Пример. Имеется множество из 4-х элементов. Необходимо получить все возможные размещения по 2 элемента.

Порядковый номер	Запись в двоичном коде				Запись в общем виде
0	0	0	0	0	$\{\varnothing\}$
1	0	0	0	1	$\{A_4\}$
2	0	0	1	0	$\{A_3\}$
3	0	0	1	1	$\{A_3, A_4\}$
4	0	1	0	0	$\{A_2\}$
5	0	1	0	1	$\{A_2, A_4\}$
6	0	1	1	0	$\{A_2, A_3\}$
7	0	1	1	1	$\{A_2, A_3, A_4\}$
8	1	0	0	0	$\{A_1\}$
9	1	0	0	1	$\{A_1, A_4\}$
10	1	0	1	0	$\{A_1, A_3\}$
11	1	0	1	1	$\{A_1, A_3, A_4\}$
12	1	1	0	0	$\{A_1, A_2\}$
13	1	1	0	1	$\{A_1, A_2, A_4\}$
14	1	1	1	0	$\{A_1, A_2, A_3\}$
15	1	1	1	1	$\{A_1, A_2, A_3, A_4\}$

$$\{A_3, A_4\} \qquad \{A_4, A_3\}$$

$$\{A_2, A_4\} \qquad \{A_4, A_2\}$$

$$\{A_2, A_3\} \qquad \{A_3, A_2\}$$

$$\{A_1, A_4\} \qquad \{A_4, A_1\}$$

$$\{A_1, A_3\} \qquad \{A_3, A_1\}$$

$$\{A_1, A_2\} \qquad \{A_2, A_1\}$$

$$\left|A_{m,B}\right| = A_n^m = \frac{n!}{(n-m)!} \qquad m = 2 \qquad \left|A_{2,B}\right| = A_4^2 = \frac{4!}{(4-2)!} = 12$$

Генерация размещений

Реализация генератора размещений на языке C++.

Как и в предыдущих случаях, генератор размещений тоже реализован в виде структуры. Структура **accomodation** имеет один конструктор. С помощью двух параметров конструктору передается размерность исходного множества (параметр **n**) и размерность генерируемых размещений (параметр **m**).

В своей работе генератор размещений использует два встроенных генератора: сочетаний (указатель **cgen**) и перестановок (указатель **pgen**).

Генерация размещений

Реализация генератора размещений на языке C++.

```
 accomodation::accomodation
(short n, short m)
  {
   this->n = n;
   this->m = m;
   this->cgen = new
xcombination(n,m);
   this->pgen = new
permutation(m);
   this->sset = new short[m];
   this->reset();
  }
```

Генерация размещений

Реализация генератора размещений на языке C++.

Текущее состояние генератора определяется состоянием используемых генераторов, а также значением четырех переменных: **n** (количество элементов исходного множества), **m** (размерность генерируемых размещений), **sset** (указатель на массив индексов) и **na** (номер текущего размещения). Все переменные, включая указатели генераторов, инициализируются в конструкторе. Значение **na** увеличивается на единицу после генерации очередного размещения. Элементы массива **sset** меняются при каждом цикле работы генератора.

Генерация размещений

Реализация генератора размещений на языке C++.

```
 accomodation::accomodation
(short n, short m)
  {
   this->n = n;
   this->m = m;
   this->cgen = new
xcombination(n,m);
   this->pgen = new
permutation(m);
   this->sset = new short[m];
   this->reset();
  }
void  accomodation::reset()
  {
   this->na = 0;
```

Генерация размещений

Реализация генератора размещений на языке C++.

Кроме конструктора, структура **accomodation** содержит еще пять функций.

Функция **getfirst** не имеет параметров и предназначена для формирования первого размещения. Первое размещение совпадает с первым сочетанием, сформированным функцией **getfirst** встроенного генератора сочетаний.

```
short accomodation::getfirst()
 {
  short rc = (this->n >= this->m)?this->m:-1;
  if (rc > 0)
  {
   for (int i = 0; i <= this->m; i++)
   this->sset[i] = this->cgen->sset[this->pgen->ntx(i)];
  };
 return rc;
 };
```

Генерация размещений

Реализация генератора размещений на языке C++.

Функция **reset** позволяет сбросить текущее состояние генератора для того, чтобы начать его работу сначала. Функция выполняет сброс встроенных генераторов, устанавливает значение нумератора размещений (переменная **na**) в нуль и выполняет функцию **getfirst** встроенного генератора сочетаний.

```
void  accomodation::reset()
 {
  this->na = 0;
  this->cgen->reset();
  this->pgen->reset();
  this->cgen->getfirst();
 };
```

Генерация размещений

Реализация генератора размещений на языке С++.

Функция **getnext** формирует массив индексов следующего размещения и увеличивает значение переменной **na** на единицу. В своей работе функция использует функции **getnext** или **getfirst** встроенных генераторов, а также функцию **getfirst** генератора размещений.

```
short accomodation::getnext()
 {
  short rc;
  this->na++;
  if ((this->pgen->getnext())> 0) rc = this-
>getfirst();
  else  if ((rc = this->cgen->getnext())> 0)
    {this->pgen->reset();  rc = this-
>getfirst();};
  return rc;
 };
```

Генерация размещений

Реализация генератора размещений на языке С++.

Функция **ntx** возвращает значение элемента массива индексов по индексу этого элемента и служит для сокращения записи при переборе элементов массива.

Функция **count** вычисляет и возвращает общее количество размещений из **n** по **m** элементов.

```
short accomodation::ntx(short i)
 { return this->sset[i];  };

 unsigned __int64 fact(unsigned __int64 x){
return (x == 0)?1:(x*fact(x-1));};

 unsigned __int64  accomodation::count()
const
 {
return (this->n >= this->m)?
      fact(this->n)/fact(this->n - this->m):0;
 };
```

```
// --- main
#include "stdafx.h"
#include <iostream>
#include <iomanip>
#include "Combi.h"
#define N (sizeof(AA)/2)
#define M 3
int _tmain(int argc, _TCHAR* argv[])
{
 setlocale(LC_ALL, "rus");
 char  AA[][2]= {"A", "B", "C", "D"};
 std::cout<<std::endl<<" --- Генератор размещений ---";
 std::cout<<std::endl<<"Исходное множество: ";
 std::cout<<"{ ";
 for (int i = 0; i < N; i++)
   std::cout<<AA[i]<<((i< N-1)?", ":" ");
 std::cout<<"}";
```

```
std::cout<<std::endl<<"Генерация размещений  из  "<< N <<" по "<<M;
 combi::accomodation s(N,M);
 int  n  = s.getfirst();
 while (n >= 0)
 {
 std::cout<<std::endl<<std::setw(2)<<s.na<<": { ";
 for (int i = 0; i < 3; i++)
 std::cout<<AA[s.ntx(i)]<<((i< n-1)?", ":" ");
 std::cout<<"}";
 n = s.getnext();
 };
 std::cout<<std::endl<<"всего: "<<s.count()<<std::endl;
 system("pause");
 return 0;
}
```

g:\Documents and Settings\smw\Мои документы...

```
 --- Генератор размещений ---
Исходное множество: { A, B, C, D }
Генерация размещений  из  4 по 3
 0: { A, B, C }
 1: { A, C, B }
 2: { C, A, B }
 3: { C, B, A }
 4: { B, C, A }
 5: { B, A, C }
 6: { A, B, D }
 7: { A, D, B }
 8: { D, A, B }
 9: { D, B, A }
10: { B, D, A }
11: { B, A, D }
12: { A, C, D }
13: { A, D, C }
14: { D, A, C }
15: { D, C, A }
16: { C, D, A }
17: { C, A, D }
18: { B, C, D }
19: { B, D, C }
20: { D, B, C }
21: { D, C, B }
22: { C, D, B }
23: { C, B, D }
всего: 24
Для продолжения нажмите любую клавишу . . .
```

Решение задачи об оптимальном размещении контейнеров на судне с помощью генератора размещений

На палубе судна имеется m мест для размещения грузовых контейнеров. Выбрать m контейнеров для погрузки на судно можно из $n \geq m$ имеющихся в наличии. Каждый контейнер i характеризуется весом v_i и доходом c_i, $i = \overline{1, n}$ от его перевозки. На каждое место $i = \overline{1, m}$ можно разместить контейнер, если его вес $\breve{v}_i \leq v_i \leq \widehat{v}_i$, где $\breve{v}_i$ и $\widehat{v}_i$ – заданные величины.

Необходимо выбрать m контейнеров из n имеющихся таким образом, чтобы доход от перевозки был максимально возможным.

Математическая модель задачи

$$\sum_{i=1}^{m} c_{k_i} \to \max, \ \breve{v}_{k_i} \leq v_{k_i} \leq \widehat{v}_{k_i}, \ k_i \in \{1, 2, \ldots, n\}, \ \forall (i \neq j)$$
$$k_i \neq k_j, \quad i = \overline{1, m}.$$

Каждый элемент этого вектора может принимать целое значение из отрезка $\boldsymbol{X}$ и при этом все значения k_i должны быть разными.

Решение задачи об оптимальном размещении контейнеров на судне с помощью генератора размещений

Решением задачи будет вектор $(k_1, k_2, \ldots, k_m)$.

Каждый элемент этого вектора может принимать целое значение из отрезка $[1,n]$ и при этом все значения k_i должны быть разными.

Задача имеет следующие исходные данные:

n=4 – общее количество контейнеров;

m=3 – количество свободных мест на палубе судна;

{100, 200, 300, 400} – вес контейнеров

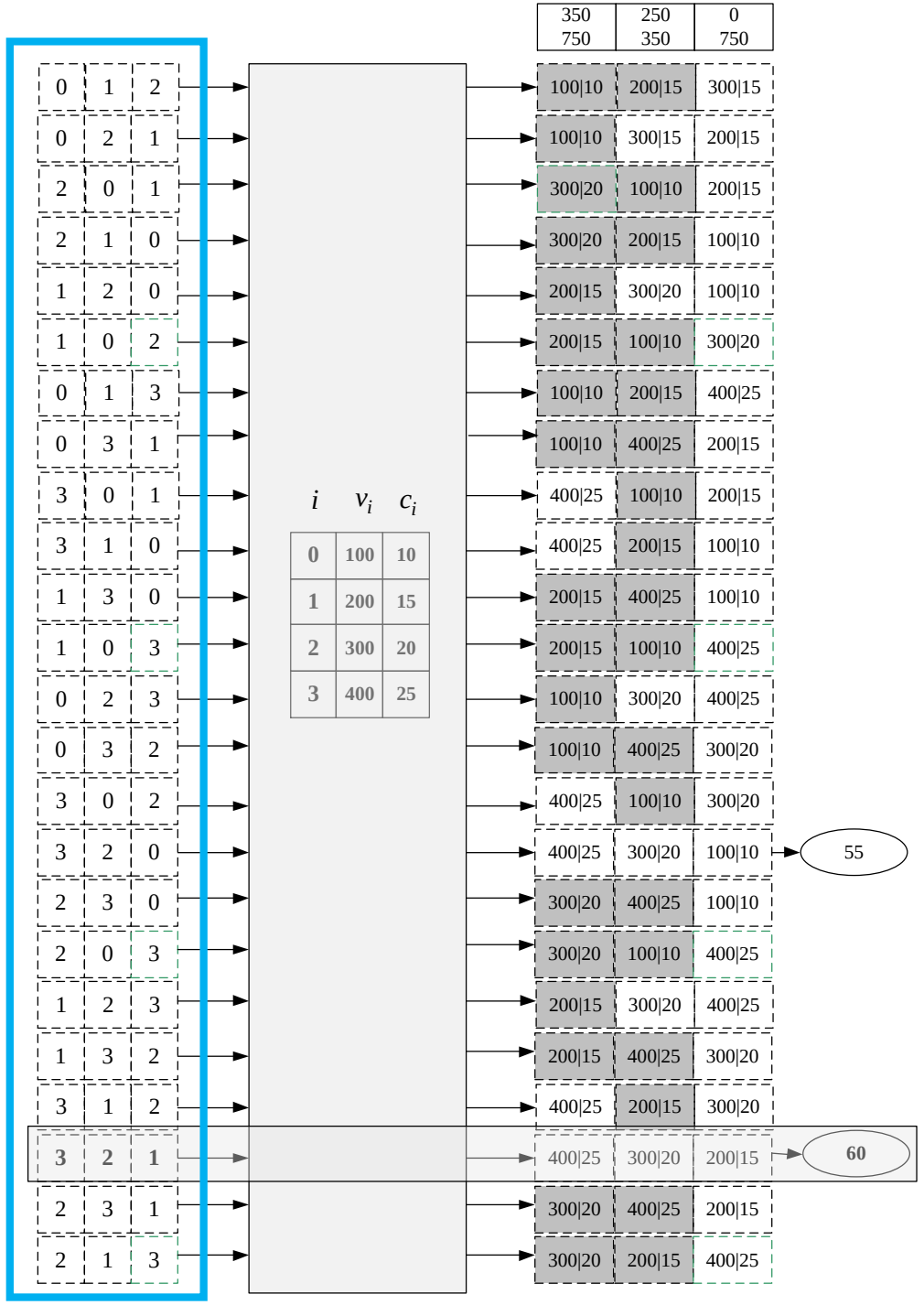
{10, 15, 20, 25} – доход от перевозки контейнеров

{350, 250, 0} – минимальный вес контейнеров

{750, 350, 750} – максимальный вес контейнеров

Решение задачи об оптимальном размещении контейнеров на судне с помощью генератора размещений

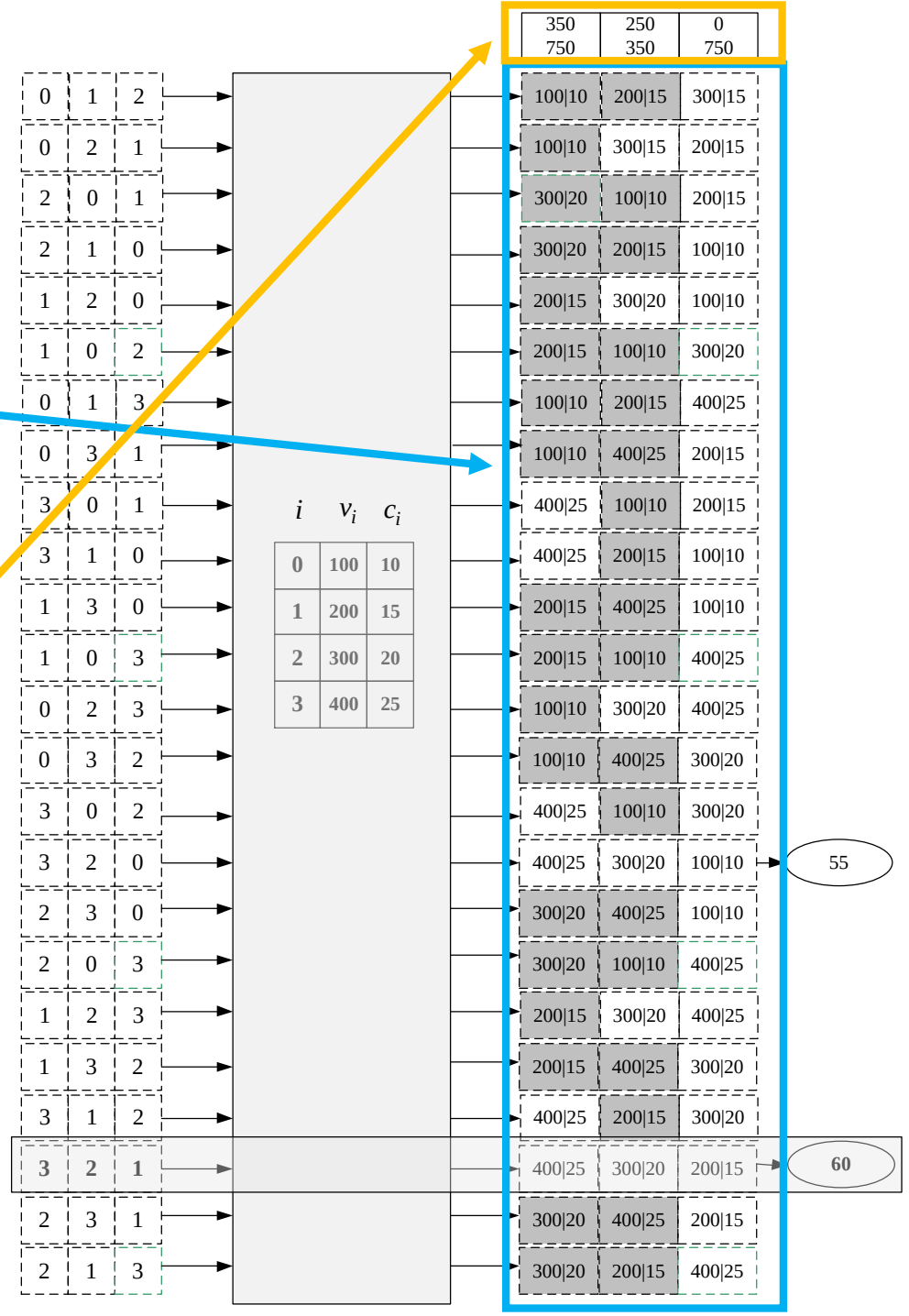
Справа на схеме показана таблица, содержащая все размещения по три элемента множества {0, 1, 2, 3}. *Эти размещения могут быть получены с помощью генератора размещений.*



Решение задачи об оптимальном размещении контейнеров на судне с помощью генератора размещений

Строки таблицы, представленной на схеме справа, содержат планы всех размещений контейнеров по свободным местам на палубе судна.

При этом в первой заголовочной строке таблицы указаны ограничения на вес контейнера для соответствующего места (два числа, обозначающие минимальный и максимальный вес), а в остальных ячейках для каждого места приведены вес соответствующего контейнера и доход от его перевозки в формате $v_i | c_i$.

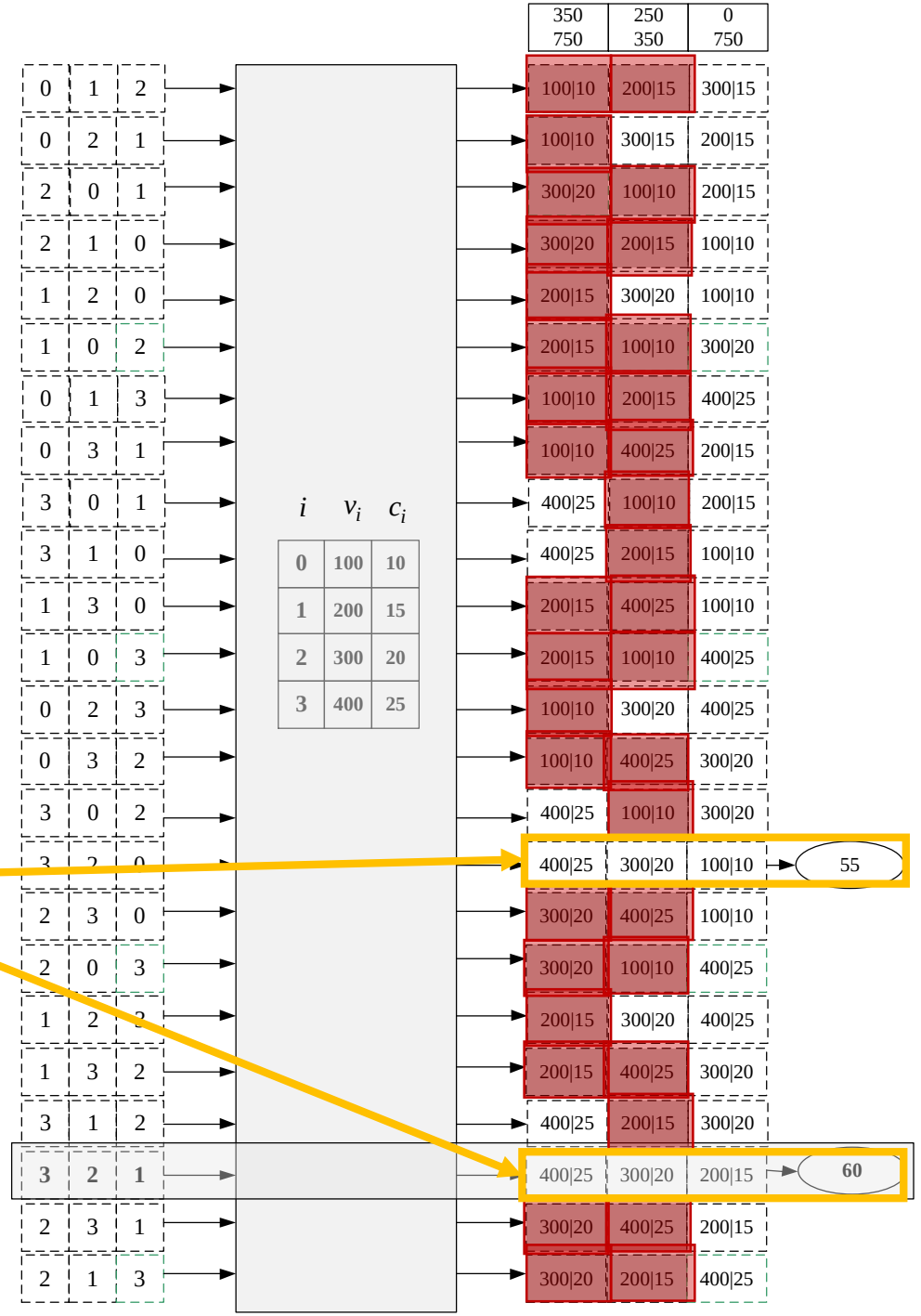


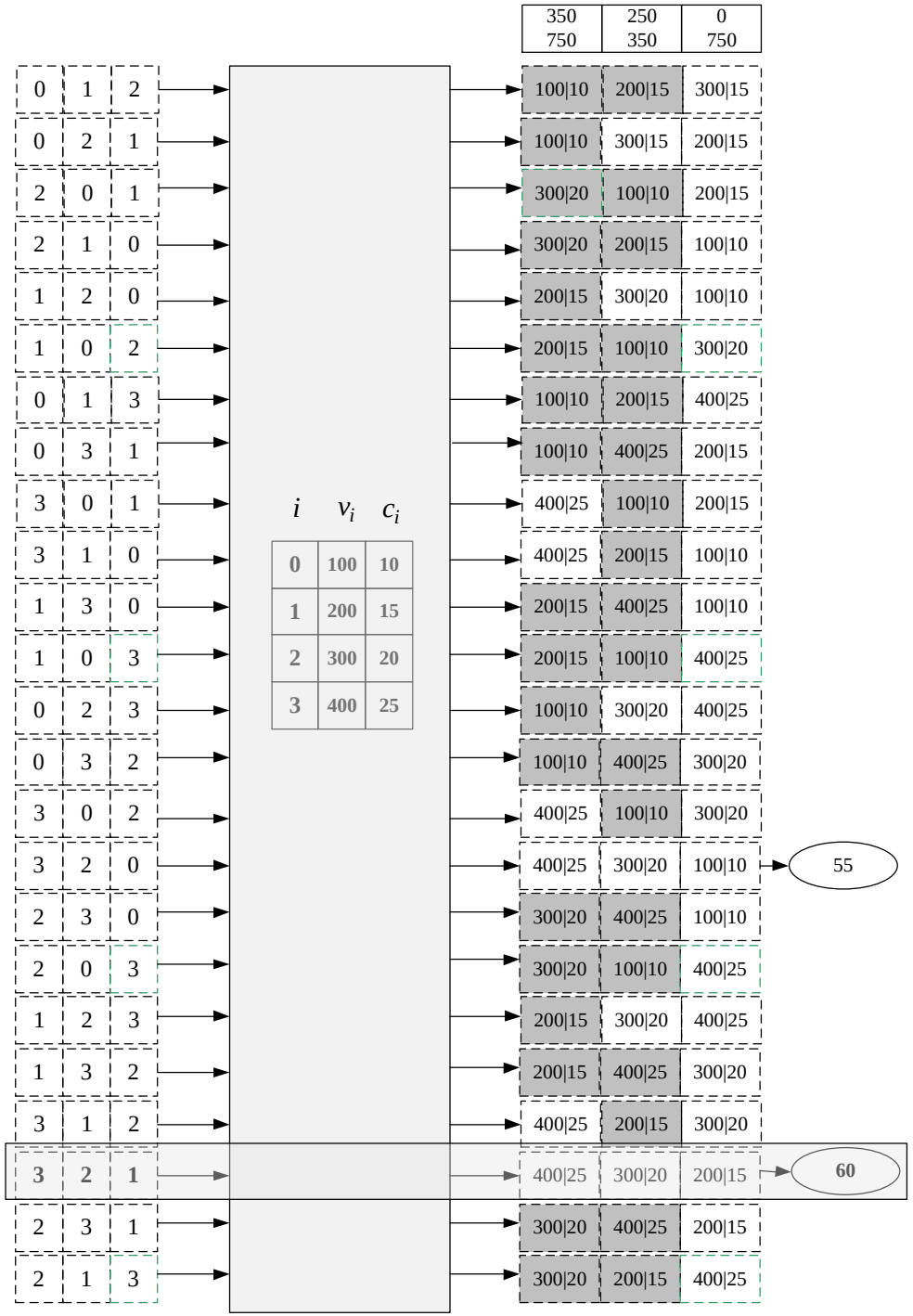
Решение задачи об оптимальном размещении контейнеров на судне с помощью генератора размещений

Закрашенные ячейки таблицы обозначают нарушение ограничений по весу контейнеров для соответствующих мест. План размещения, имеющий хотя бы одну закрашенную ячейку, является недопустимым.

На схеме только два плана размещения контейнеров являются допустимыми. Для этих планов вычисляется доход от перевозки выбранных контейнеров. Доход указан в овальной рамке рядом со строкой плана.

Оптимальный план размещения контейнеров на палубе судна выделен рамкой.





Количество различных способов разместить контейнеры на палубе равно:

$$\left|A_{m,B}\right| = A_n^m = \frac{n!}{(n-m)!},$$

$$A_4^3 = \frac{4!}{(4-3)!} = 24.$$

Элементы размещений, полученных с помощью генератора используются в качестве индексов в массивах, содержащих вес и доход от транспортировки контейнеров.

Решение задачи об оптимальном размещении контейнеров на судне с помощью генератора размещений (*пример реализации на языке C++*)

Функция **boat_c** имеет семь параметров, определяющих условие задачи:

1. **m** - количество мест для установки контейнеров,
2. **minv** - массив размерностью **m**, содержащий минимальный вес контейнера для каждого места,
3. **maxv** - массив размерностью **m**, содержащий максимальный вес контейнера для каждого места,
4. **n** - количество контейнеров,
5. **v** - массив размерностью **n**, содержащий вес контейнеров,
6. **c** - массив размерностью **n**, содержащий величину дохода от перевозки каждого контейнера)
7. **r** - массив размерностью **m**, содержащий номера выбранных контейнеров (возвращаемый параметр) .

Если решение задачи существует, то функция возвращает положительное значение, равное величине дохода, иначе возвращается нуль.

Решение задачи об оптимальном размещении контейнеров на судне с помощью генератора размещений

(*пример реализации на языке C++*)

```
int boat_c(
        short m,
        int minv[],
        int maxv[],
        short n,
        const int v[],
        const int c[],
        short r[]);
```

Решение задачи об оптимальном размещении контейнеров на судне с помощью генератора размещений (*пример реализации на языке C++*)

В функции **boat_c** применяется генератор размещений (тип **combi::accomodation**). Конструктору генератора передается два параметра: **n** (количество контейнеров) и **m** (количество мест для установки контейнеров).

```
int boat_c(short m, int minv[], int maxv[], short n, const int v[], const int c[], short r[] )
 {
 combi::accomodation s(n, m);
 int rc = 0, i = s.getfirst(), cc = 0;
 while (i > 0)
 { if (boatfnc::compv(s, minv, maxv, v))
   if ((cc = boatfnc::calcc(s,c)) > rc)
     {rc = cc; boatfnc::copycomb(m, r, s.sset);}
         i = s.getnext(); };
         return rc;};
```

Решение задачи об оптимальном размещении контейнеров на судне с помощью генератора размещений (*пример реализации на языке C++*)

Кроме того, функция вызывает три вспомогательные функции:

1. Функция **boatfnc::compv** позволяет проверить допустимость размещения контейнеров. Если вес всех контейнеров в размещении удовлетворяет заданным ограничениям, то функция возвращает **true**, иначе возвращается **false**.

```
bool compv( combi::accomodation s, const int ming[],
        const int maxg[], const int v[])
{
 int i = 0;
 while(i < s.m && v[s.ntx(i)] <= maxg[i] && v[s.ntx(i)] >= ming[i])i++;
 return (i == s.m);
};
```

Решение задачи об оптимальном размещении контейнеров на судне с помощью генератора размещений (*пример реализации на языке C++*)

Кроме того, функция вызывает три вспомогательные функции:

2. Функция **boatfnc::calcc** дает возможность вычислить доход от перевозки контейнеров при заданном размещении.

```
int calcc(combi::accomodation s, const int c[])
{
 int rc = 0;
 for (int i = 0; i < s.m; i++) rc += c[s.ntx(i)];
 return rc;
};
```

Решение задачи об оптимальном размещении контейнеров на судне с помощью генератора размещений (*пример реализации на языке C++*)

Кроме того, функция вызывает три вспомогательные функции:

3. Функция **boatfnc::copycomb** предназначена для сохранения (копирования) наиболее доходного на данном шаге допустимого размещения.

```
void copycomb(short m, short *r1, const short *r2)
 { for (int i = 0; i <  m; i++)  r1[i] = r2[i]; };
}
```

Решение задачи об оптимальном размещении контейнеров на судне с помощью генератора размещений (*пример реализации на языке C++*)

Функция **boat_c** в цикле генерирует все возможные размещения контейнеров (функции **getfirst** и **getnext** генератора размещений), проверяет их допустимость (функция **boatfnc::compv**), вычисляет для допустимых размещений доход от перевозки контейнеров (функция **boatfnc::calcc**), фиксирует оптимальное размещение (функция **boatfnc::copycomb**) и возвращает доход от оптимального размещения или нуль, если решения нет.

Решение задачи об оптимальном размещении контейнеров на судне с помощью генератора размещений

(*пример реализации на языке C++*)

```
int boat_c(short m, int minv[], int maxv[],
short n, const int v[], const int c[], short r[] )
 {
 combi::accomodation s(n, m);
 int rc = 0, i = s.getfirst(), cc = 0;
 while (i > 0)
 { if (boatfnc::compv(s, minv, maxv, v))
   if ((cc = boatfnc::calcc(s,c)) > rc)
     {rc = cc; boatfnc::copycomb(m, r,
s.sset);}
          i = s.getnext(); };
          return rc;};
```

Решение задачи об оптимальном размещении контейнеров на судне с помощью генератора размещений (*пример реализации на языке C++*)

Программа сначала подготавливает массивы данных: **v** (вес каждого контейнера), **c** (доход от перевозки каждого контейнера), **minv** (нижняя граница веса контейнера для каждого свободного места на палубе судна), **maxv** (верхняя граница веса контейнера для каждого свободного места), а затем вызывает функцию **boat_c**.

В результате выполнения функция возвращает доход от оптимального размещения контейнеров на палубе судна, а также параметр (массив **r**), содержащий перечень номеров выбранных контейнеров.

Решение задачи об оптимальном размещении контейнеров на судне с помощью генератора размещений

(*пример реализации на языке C++*)

```
#define NN (sizeof(v)/sizeof(int))
#define MM 3
int _tmain(int argc, _TCHAR* argv[])
{
 setlocale(LC_ALL, "rus");
 int v[] =   {100,  200, 300,  400};
int c[] =   { 10, 15,  20, 25};
int minv[]  = {350,  250,  0};
int maxv[]  = {750,  350,  750};
short r[MM];
 int cc = boat_c( ……){…};
```

```
--- Задача о размещении контейнеров на судне ---
- общее количество контейнеров       : 4
- количество мест для контейнеров    : 3
- минимальный  вес контейнера        : 350 250   0
- максимальный вес контейнера        : 750 350 750
- вес контейнеров                    : 100 200 300 400
- доход от перевозки                 :  10  15  20  25
- выбраны контейнеры <0,1,...,m-1>   : 3 2 1
- доход от перевозки                 : 60

Для продолжения нажмите любую клавишу . . .
```

Решение задачи об оптимальном размещении контейнеров на судне с помощью генератора размещений (*пример реализации на языке C++*)

Далее представлена программа, позволяющая оценить продолжительность решения задачи о размещении контейнеров в зависимости от количества свободных мест на палубе судна и результат ее выполнения.

Здесь применяются функции **auxil::start** и **auxil::iget**, позволяющие сгенерировать случайные последовательности целых чисел в заданных пределах.

```
…
#define SPACE(n) std::setw(n)<<" "
#define NN 9
int _tmain(int argc, _TCHAR* argv[])
{
 setlocale(LC_ALL, "rus");
 int v[NN+1], c[NN+1], minv[NN+1], maxv[NN+1];
 short r[NN];
 auxil::start();
 for(int i = 0; i <= NN; i++)
 {
 v[i] = auxil::iget(50,500); c[i] = auxil::iget(10,30);
 minv[i] = auxil::iget(50,300); maxv[i] = auxil::iget(250,750);
 }
```

```
std::cout<<std::endl<<"-- Задача о размещении контейнеров -- ";
 std::cout<<std::endl<<"-- всего контейнеров: " << NN;
 std::cout<<std::endl<<"-- количество ------ продолжительность -- ";
 std::cout<<std::endl<<"  мест     вычисления  ";
 clock_t t1, t2;
 for (int i = 4; i < NN; i++)
 {
 t1 = clock();
 boat_c(i, minv,  maxv, NN,  v,  c, r);
 t2 = clock();
 std::cout<<std::endl<<SPACE(7)<<std::setw(2)<<i
      <<SPACE(15)<<std::setw(6)<<(t2-t1);
 }
 std::cout<<std::endl; system("pause");
 return 0;
}
```

```
-- Задача о размещении контейнеров --
-- всего контейнеров: 11
-- количество ------ продолжительность --
    мест                вычисления
       4                         0
       5                        47
       6                       312
       7                      1641
       8                      6953
       9                     22094
      10                     46437

Для продолжения нажмите любую клавишу . . .
```

График, отражающий зависимость продолжительности решения задачи от количества свободных мест на палубе судна при общем количестве контейнеров, равном 11. Вид графика вполне согласуется с оценкой сложности алгоритма генерации размещений ***n*** из 11 элементов .

Сложность $O(A_m^n)$